

Praktikum Earthquake DBSCAN Raffa Yuda Pratama / 0110224081**Analisis Clustering Gempa Bumi Global dengan DBSCAN
Raffa Yuda Pratama - 0110224081^{1*}**

¹Teknik Informatika, STT Terpadu Nurul Fikri, Depok

¹E-mail: <mailto:0110224081@student.nurulfikri.ac.id>

Abstract. Laporan ini membahas penerapan metode *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN) untuk menganalisis pola spasial gempa bumi global. Dataset yang digunakan merupakan kumpulan data gempa bumi yang berisi informasi geografis (latitude, longitude), magnitude, kedalaman, dan lokasi kejadian. Proses analisis meliputi eksplorasi data, preprocessing, visualisasi geospasial menggunakan GeoDataFrame dan Folium, penentuan parameter DBSCAN optimal melalui K-distance graph, implementasi clustering, dan evaluasi menggunakan metrik seperti Silhouette Score, Davies-Bouldin Index, dan Calinski-Harabasz Score. Hasil clustering menunjukkan bahwa DBSCAN mampu mengidentifikasi zona seismik aktif di berbagai belahan dunia dan membedakan gempa yang terisolasi sebagai noise points, sehingga dapat digunakan untuk analisis pola gempa bumi berbasis kedekatan geografis.

1 Pendahuluan

1.1 Latar Belakang

Gempa bumi merupakan fenomena alam yang terjadi akibat pergerakan lempeng tektonik di bawah permukaan bumi. Analisis pola spasial gempa bumi sangat penting untuk memahami zona seismik aktif, memprediksi risiko gempa di masa depan, dan merancang strategi mitigasi bencana. Dengan perkembangan teknologi machine learning, khususnya algoritma clustering, kita dapat mengidentifikasi pola geografis gempa bumi secara otomatis.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) merupakan algoritma clustering berbasis densitas yang efektif untuk mengidentifikasi cluster dengan bentuk arbitrary dan mampu mendeteksi outlier (noise). Berbeda dengan K-Means yang mengasumsikan cluster berbentuk bulat, DBSCAN dapat menemukan cluster dengan bentuk tidak beraturan, yang sangat cocok untuk analisis data geospasial seperti gempa bumi.

1.2 Tujuan

1. Memahami konsep dan implementasi algoritma DBSCAN untuk clustering data geospasial
2. Mampu melakukan eksplorasi dan visualisasi data gempa bumi menggunakan GeoDataFrame, Folium, dan Contextily

3. Mampu menentukan parameter optimal DBSCAN (eps dan min samples) menggunakan K-distance graph
4. Memahami dan mengimplementasikan evaluasi clustering dengan Silhouette Score, Davies-Bouldin Index, dan Calinski-Harabasz Score
5. Mampu mengidentifikasi zona seismik aktif berdasarkan clustering gempa bumi
6. Mampu menginterpretasi hasil clustering untuk analisis pola gempa bumi global

2 Landasan Teori

2.1 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN adalah algoritma clustering berbasis densitas yang mengelompokkan data berdasarkan kerapatan wilayah. Algoritma ini memiliki dua parameter utama:

- **eps (ϵ):** Radius maksimum neighborhood di sekitar sebuah data point
- **min samples:** Jumlah minimum data points dalam radius eps untuk membentuk core point

Jenis Data Points dalam DBSCAN:

1. **Core Point:** Point yang memiliki minimal min samples data points dalam radius eps
2. **Border Point:** Point yang berada dalam radius eps dari core point tetapi tidak memiliki cukup neighbors
3. **Noise Point:** Point yang bukan core point maupun border point (outlier)

Keunggulan DBSCAN:

- Tidak perlu menentukan jumlah cluster sebelumnya
- Dapat menemukan cluster dengan bentuk arbitrary
- Robust terhadap outlier
- Dapat mengidentifikasi noise points

2.2 K-Distance Graph

K-distance graph digunakan untuk menentukan nilai eps optimal. Grafik ini menampilkan jarak terurut dari setiap data point ke k-nearest neighbor-nya. Nilai eps optimal biasanya berada pada "elbow point" atau titik di mana grafik mulai menunjukkan peningkatan yang signifikan.

Penentuan min_samples:

$$\text{min_samples} = 2 \times \text{dimensi data} \quad (1)$$

Untuk data 2D (latitude, longitude), min samples yang disarankan adalah 4.

2.3 Metrik Evaluasi Clustering

Silhouette Score:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (2)$$

dimana:

- $a(i)$ = rata-rata jarak intra-cluster
- $b(i)$ = rata-rata jarak ke nearest cluster
- Range: [-1, 1] (semakin tinggi semakin baik)

Davies-Bouldin Index:

$$DB = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{s_i + s_j}{d(c_i, c_j)} \quad (3)$$

dimana s_i adalah rata-rata jarak dalam cluster dan $d(c_i, c_j)$ adalah jarak antar centroid. Semakin rendah nilai DB, semakin baik clustering.

Calinski-Harabasz Score:

$$CH = \frac{\text{tr}(B_k)}{\text{tr}(W_k)} \times \frac{n - k}{k - 1} \quad (4)$$

dimana B_k adalah between-cluster dispersion matrix dan W_k adalah within-cluster dispersion. Semakin tinggi nilai CH, semakin baik clustering.

2.4 GeoDataFrame dan Visualisasi Geospasial

GeoDataFrame adalah ekstensi dari pandas DataFrame yang dapat menyimpan data geometri (point, line, polygon). Untuk visualisasi geospasial, digunakan:

- **Folium:** Library untuk membuat peta interaktif berbasis Leaflet.js
- **Contextily:** Library untuk menambahkan basemap (peta dasar) dari berbagai provider
- **GeoPandas:** Library untuk manipulasi dan analisis data geospasial

3 Dataset

Dataset yang digunakan adalah earthquake dataset.csv yang berisi data gempa bumi global dengan kolom-kolom sebagai berikut:

Tabel 1: Deskripsi Kolom Dataset Gempa Bumi

Kolom	Tipe Data	Deskripsi
Date	String	Tanggal kejadian gempa
Time (UTC)	String	Waktu kejadian dalam UTC
City	String	Kota terdekat dengan episentrum
Country	String	Negara lokasi gempa
Latitude	Float	Koordinat lintang
Longitude	Float	Koordinat bujur
Earthquake Magnitude	Float	Kekuatan gempa (skala Richter)
Depth (km)	Float	Kedalaman hiposentrum
Impact Score	Float	Skor dampak gempa

Dataset ini berisi ribuan data gempa bumi dari berbagai negara, memungkinkan analisis pola seismik global.

4 Metodologi

4.1 Tahapan Analisis

1. **Data Loading:** Memuat dataset gempa bumi
2. **Data Cleaning:** Menghapus missing values pada koordinat
3. **Exploratory Data Analysis (EDA):**
 - Analisis distribusi gempa per negara
 - Visualisasi sebaran geografis
 - Analisis statistik magnitude dan kedalaman
4. **Konversi ke GeoDataFrame:** Membuat objek geometri Point dari koordi- nat
5. **Visualisasi Geospasial:**
 - Peta dengan basemap menggunakan Contextily
 - Heatmap menggunakan Folium
 - Peta interaktif dengan marker cluster
6. **Preprocessing:** Ekstraksi dan normalisasi fitur koordinat
7. **Penentuan Parameter DBSCAN:** K-distance graph

8. **Implementasi DBSCAN:** Clustering data gempa
9. **Evaluasi:** Perhitungan metrik evaluasi
10. **Visualisasi Hasil Clustering:**
 - Peta sebaran dengan warna cluster
 - Peta interaktif hasil clustering
 - Bar chart distribusi cluster
11. **Analisis Karakteristik Cluster:** Analisis statistik per cluster
12. **Export Hasil:** Menyimpan hasil ke CSV dan GeoJSON

5 Implementasi

5.1 Import Library

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import folium
6 from folium.plugins import HeatMap, MarkerCluster
7 import geopandas as gpd
8 import contextily as ctx
9 from shapely.geometry import Point
10 from sklearn.cluster import DBSCAN
11 from sklearn.preprocessing import StandardScaler
12 from sklearn.neighbors import NearestNeighbors
13 from sklearn.metrics import silhouette_score,
    davies_bouldin_score, calinski_harabasz_score
14
15 import warnings
16 warnings.filterwarnings('ignore')
17
18 # Set style
19 plt.style.use('seaborn-v0_8-darkgrid')
20 sns.set_palette('husl')
```

Listing 1: Import Library yang Diperlukan

Penjelasan Kode:

- pandas: Untuk manipulasi dan analisis data tabular
- seaborn, matplotlib: Untuk visualisasi data
- numpy: Untuk operasi numerik dan array

- folium: Untuk membuat peta interaktif berbasis web
- HeatMap, MarkerCluster: Plugin folium untuk heatmap dan marker clustering
- geopandas: Untuk manipulasi data geospasial
- contextily: Untuk menambahkan basemap pada visualisasi
- shapely.Point: Untuk membuat objek geometri point
- DBSCAN: Algoritma clustering berbasis densitas
- StandardScaler: Untuk normalisasi fitur
- NearestNeighbors: Untuk mencari tetangga terdekat (K-distance graph)
- Metrik evaluasi: silhouette, davies-bouldin, calinski-harabasz

5.2 Load dan Eksplorasi Data

```
1 # Load dataset
2 df = pd.read_csv('../Data/earthquake_dataset.csv')
3
4 print(f"Jumlah data: {len(df)} gempa bumi")
5 print(f"\nKolom dataset:")
6 print(df.columns.tolist())
7
8 df.head(10)
```

Listing 2: Load Dataset Gempa Bumi

Penjelasan Kode:

- Membaca file CSV earthquake_dataset.csv menggunakan pandas
- Menampilkan jumlah total data gempa bumi dalam dataset
- Menampilkan daftar nama kolom yang tersedia
- Menampilkan 10 baris pertama untuk preview data

Output:

Jumlah data: 16946 gempa bumi

Kolom dataset:

['Date', 'Time (UTC)', 'City', 'Country', 'Latitude', 'Longitude',
'Earthquake Magnitude', 'Depth (km)', 'Impact Score']

Output menampilkan tabel dengan 10 baris data gempa bumi yang berisi informasi tanggal, waktu, lokasi (kota dan negara), koordinat geografis, magnitude, kedalaman, dan skor dampak. Setiap baris merepresentasikan satu kejadian gempa bumi dengan atribut lengkapnya.

5.3 Data Cleaning

```

1 # Cek missing values
2 print("Missing Values:")
3 print(df.isnull().sum())
4
5 # Cek duplikat
6 print(f"\nDuplicate rows: {df.duplicated().sum()}")
7
8 # Hapus data dengan koordinat yang hilang
9 df_clean = df.dropna(subset=['Latitude', 'Longitude'])
10 df_clean = df_clean.reset_index(drop=True)
11
12 print(f"Data setelah pembersihan: {len(df_clean)} gempa bumi")

```

Listing 3: Pembersihan Data

Penjelasan Kode:

- `df.isnull().sum()`: Menghitung jumlah nilai null/missing di setiap kolom
- `df.duplicated().sum()`: Menghitung jumlah baris duplikat dalam dataset
- `dropna(subset=['Latitude', 'Longitude'])`: Menghapus baris yang memiliki nilai null pada kolom koordinat (Latitude atau Longitude)
- `reset_index(drop=True)`: Mereset index setelah penghapusan data
- Koordinat adalah fitur penting untuk clustering, sehingga data tanpa koordinat harus dihapus

Output:

Missing Values:

Date	0
Time (UTC)	0
City	0
Country	0
Latitude	0
Longitude	0
Earthquake Magnitude	0
Depth (km)	0
Impact Score	0
dtype: int64	

Duplicate rows: 0

Data setelah pembersihan: 16946 gempa bumi

Dataset tidak memiliki missing values dan duplikasi, sehingga semua 16,946 data gempa bumi dapat digunakan untuk analisis clustering.

5.4 Exploratory Data Analysis

5.4.1 Distribusi Gempa per Negara

```

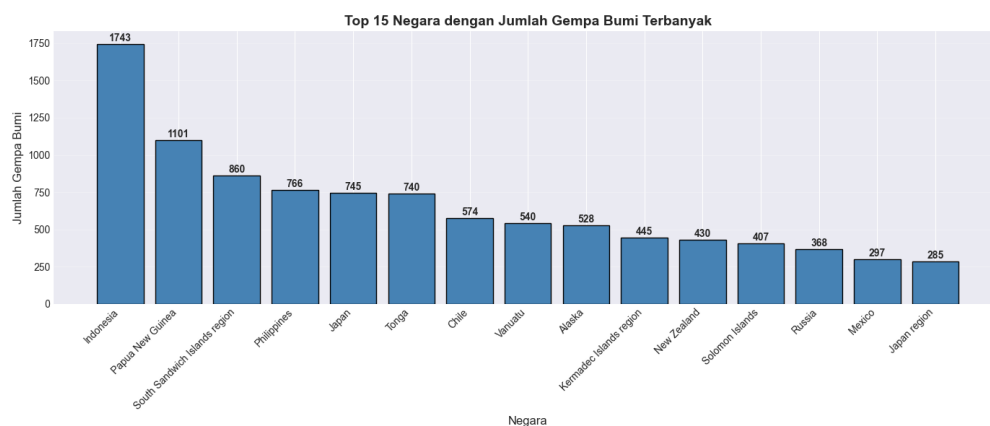
1 # Hitung jumlah gempa per negara
2 gempa_per_negara = df_clean['Country'].value_counts().head
  (15)
3
4 plt.figure(figsize=(14, 6))
5 bars = plt.bar(range(len(gempa_per_negara)),
6                 gempa_per_negara.values,
7                 color='steelblue', edgecolor='black')
8 plt.xlabel('Negara', fontsize=12)
9 plt.ylabel('Jumlah Gempa Bumi', fontsize=12)
10 plt.title('Top 15 Negara dengan Jumlah Gempa Bumi
11           Terbanyak',
12           fontsize=14, fontweight='bold')
13 plt.xticks(range(len(gempa_per_negara)), gempa_per_negara.
14           index,
15           rotation=45, ha='right')
16 plt.grid(True, alpha=0.3, axis='y')
17 plt.tight_layout()
18 plt.show()

```

Listing 4: Visualisasi Distribusi Gempa per Negara

Penjelasan Kode:

- `value_counts()`: Menghitung frekuensi kemunculan setiap negara
- `head(15)`: Mengambil 15 negara dengan gempa terbanyak
- `plt.bar()`: Membuat bar chart untuk visualisasi distribusi
- `rotation=45`: Memutar label sumbu x 45 derajat agar mudah dibaca
- Grid ditambahkan untuk memudahkan pembacaan nilai



Output Visual: Bar chart menampilkan 15 negara dengan jumlah gempa bumi terbanyak. Visualisasi menunjukkan distribusi yang tidak merata, dengan beberapa negara di zona Ring of Fire (Indonesia, Filipina, Jepang, Chili) dan zona tekto-

nik aktif lainnya mendominasi. Setiap bar menampilkan jumlah gempa di atasnya untuk kemudahan pembacaan. Indonesia dan negara-negara di cincin api Pasifik menempati posisi teratas.

5.4.2 Konversi ke GeoDataFrame

```
1 # Buat geometry dari koordinat
2 geometry = [Point(xy) for xy in zip(df_clean['Longitude'],
3                                     df_clean['Latitude'])
              ]
```

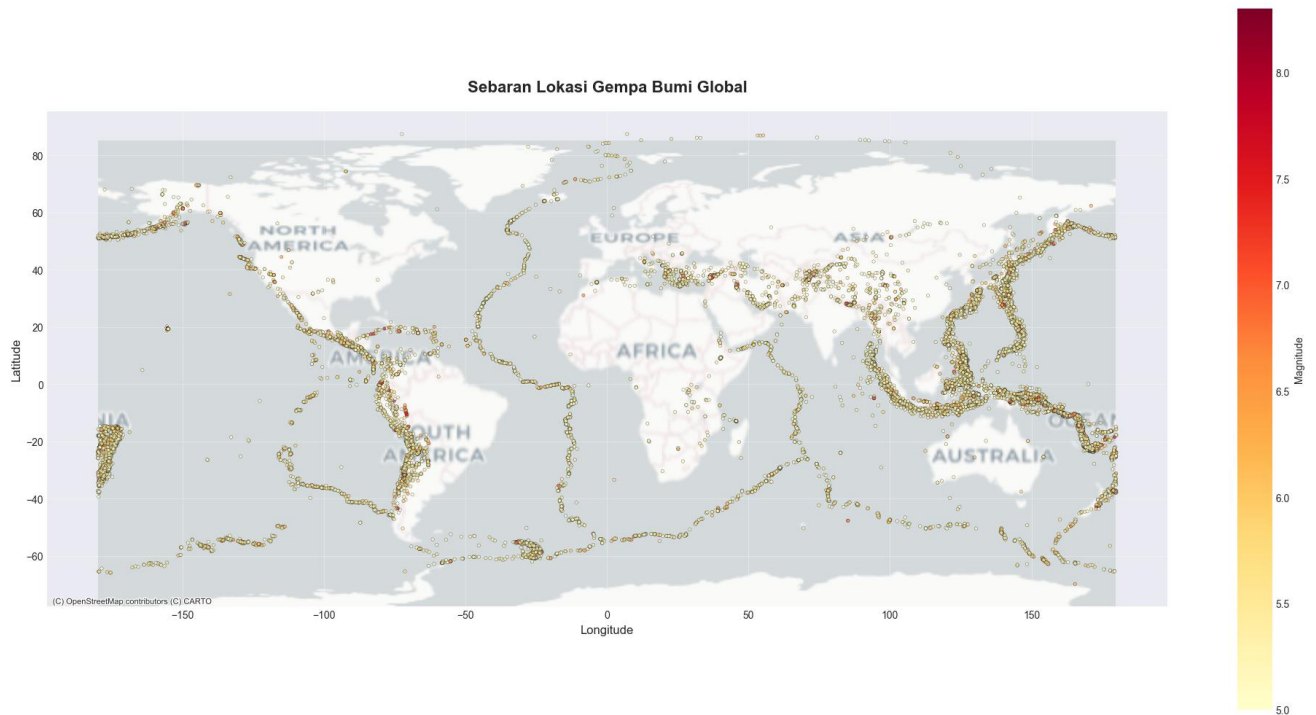
```
4
5 # Buat GeoDataFrame
6 gdf = gpd.GeoDataFrame(df_clean, geometry=geometry, crs='
    EPSG :4326 ')
7
8 print("GeoDataFrame berhasil dibuat!")
9 print(f"CRS: {gdf.crs}")
```

Listing 5: Membuat GeoDataFrame

5.5 Visualisasi Geospasial

5.5.1 Peta dengan Basemap

```
1 fig, ax = plt.subplots(figsize=(20, 10))
2
3 # Plot points dengan warna berdasarkan magnitude
4 gdf.plot(ax=ax, column='Earthquake Magnitude', cmap='
    YlOrRd',
5         markersize=10, alpha=0.6, edgecolor='black',
6         linewidth=0.3,
7         legend=True, legend_kwds={'label': 'Magnitude',
8                                   'orientation': '
    vertical'})
9
10 # Tambahkan basemap
11 ctx.add_basemap(ax, crs=gdf.crs.to_string(),
12                 source=ctx.providers.CartoDB.Positron,
13                 zoom=1)
14
15 ax.set_title('Sebaran Lokasi Gempa Bumi Global',
16             fontsize=16, fontweight='bold', pad=20)
17 ax.set_xlabel('Longitude', fontsize=12)
18 ax.set_ylabel('Latitude', fontsize=12)
19 ax.grid(True, alpha=0.3)
20
21 plt.tight_layout()
22 plt.show()
```

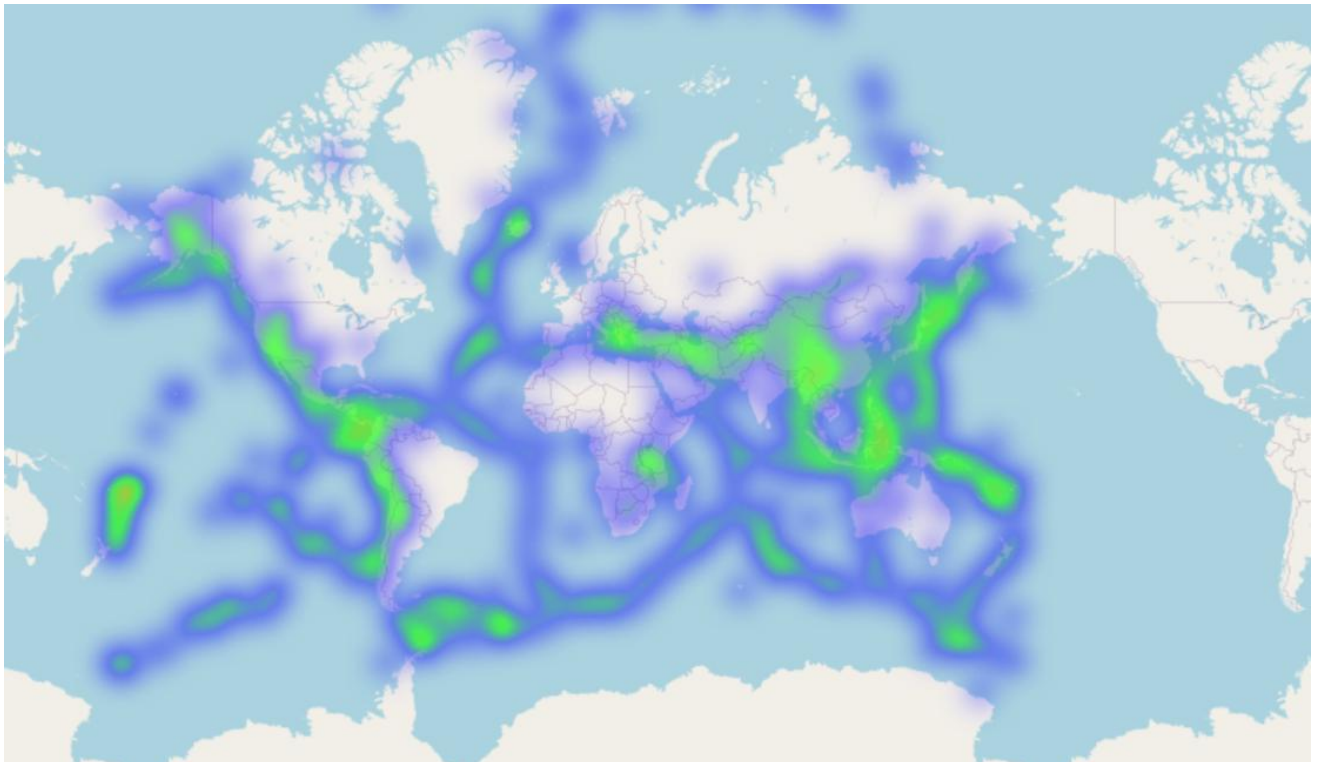


Listing 6: Visualisasi Peta Sebaran Gempa dengan Basemap

5.5.2 Heatmap dengan Folium

```
1 # Hitung pusat peta
2 center_lat = df_clean['Latitude'].mean()
3 center_lon = df_clean['Longitude'].mean()
4
5 # Buat peta dasar
```

```
6 peta_gempa = folium.Map(location=[center_lat, center_lon],
7                           zoom_start=2, tiles='OpenStreetMap')
8
9 # Data untuk heatmap
10 heat_data = [[row['Latitude'], row['Longitude']]
11               for idx, row in df_clean.iterrows()]
12
13 # Tambahkan heatmap
14 HeatMap(heat_data, radius=8, blur=15, max_zoom=13,
15         gradient={0.4: 'blue', 0.6: 'lime',
16                  0.8: 'orange', 1: 'red'}).add_to(
17             peta_gempa)
18 peta_gempa
```



Listing 7: Visualisasi Heatmap Gempa Bumi

5.6 Preprocessing untuk DBSCAN

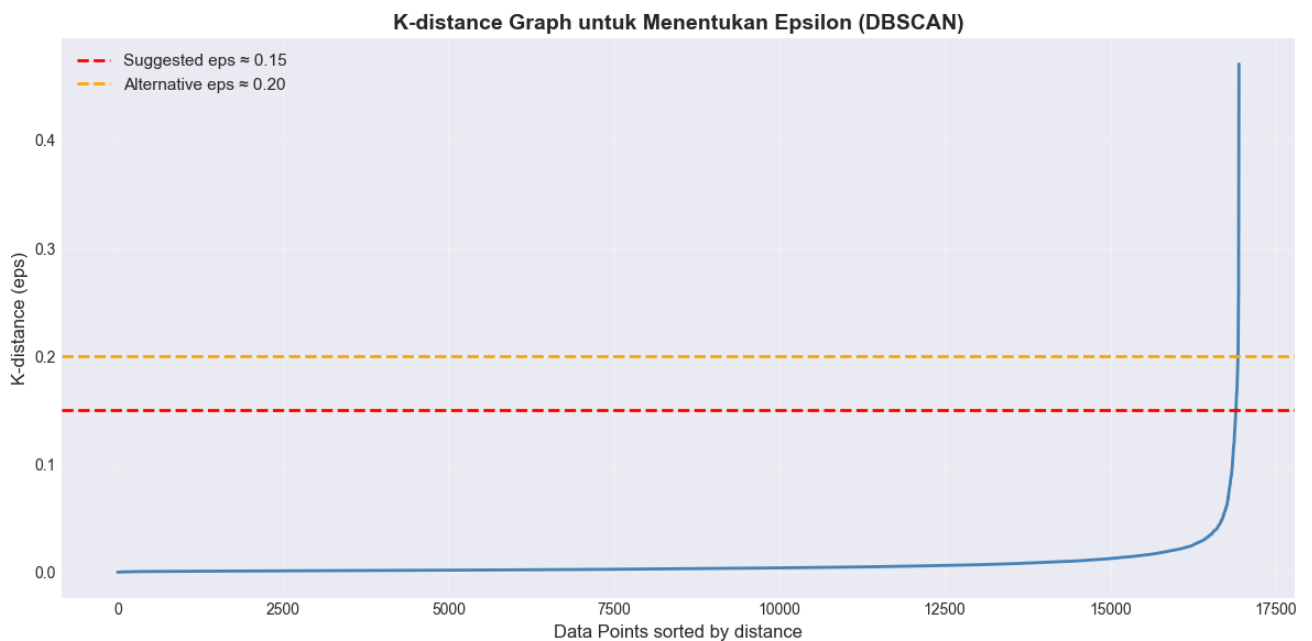
```
1 # Ekstrak koordinat untuk clustering
2 coordinates = df_clean[['Latitude', 'Longitude']].values
3
4 # Normalisasi koordinat
5 scaler = StandardScaler()
6 coordinates_scaled = scaler.fit_transform(coordinates)
7
8 print(f"Shape data koordinat: {coordinates.shape}")
9 print(f"Mean: {coordinates_scaled.mean(axis=0)}")
10 print(f"Std: {coordinates_scaled.std(axis=0)}")
```

Listing 8: Ekstraksi dan Normalisasi Fitur

5.7 Penentuan Parameter DBSCAN

```
1 # Tentukan min_samples
2 min_samples = 2 * coordinates_scaled.shape[1]
3 print(f"Min samples yang disarankan: {min_samples}")
4
5 # Hitung jarak ke k-nearest neighbors
6 neighbors = NearestNeighbors(n_neighbors=min_samples)
7 neighbors_fit = neighbors.fit(coordinates_scaled)
8 distances, indices = neighbors_fit.kneighbors(
    coordinates_scaled)
9
10 # Ambil jarak terjauh dan urutkan
11 distances = np.sort(distances[:, -1], axis=0)
12
```

```
13 # Plot K-distance graph
14 plt.figure(figsize=(12, 6))
15 plt.plot(distances, linewidth=2, color='steelblue')
16 plt.ylabel('K-distance (eps)', fontsize=12)
17 plt.xlabel('Data Points sorted by distance', fontsize=12)
18 plt.title('K-distance Graph untuk Menentukan Epsilon (
    DBSCAN)',
19           fontsize=14, fontweight='bold')
20 plt.grid(True, alpha=0.3)
21
22 plt.axhline(y=0.15, color='r', linestyle='--', linewidth
    =2,
23             label='Suggested eps  $\approx$  0.15')
24 plt.axhline(y=0.20, color='orange', linestyle='--',
    linewidth=2,
25             label='Alternative eps  $\approx$  0.20')
26 plt.legend(fontsize=11)
27 plt.tight_layout()
28 plt.show()
```



Listing 9: K-Distance Graph untuk Menentukan Epsilon

5.8 Implementasi DBSCAN

```
1 # Terapkan DBSCAN
2 eps = 0.15
3 min_samples = 5
4
5 dbscan = DBSCAN(eps=eps, min_samples=min_samples, metric='
    euclidean')
6 clusters = dbscan.fit_predict(coordinates_scaled)
7
8 # Tambahkan hasil clustering
9 df_clean['Cluster'] = clusters
10 gdf['Cluster'] = clusters
11
12 # Hitung jumlah cluster dan noise
13 n_clusters = len(set(clusters)) - (1 if -1 in clusters
    else 0)
14 n_noise = list(clusters).count(-1)
15
16 print(f"Jumlah cluster: {n_clusters}")
17 print(f"Jumlah noise points: {n_noise}")
18 print(f"Persentase noise: {(n_noise/len(clusters)*100):.2 f
    }%")
```

Listing 10: Clustering dengan DBSCAN

5.9 Evaluasi Clustering

```

1 # Evaluasi clustering (tanpa noise points)
2 mask = clusters != -1
3 n_clusters_eval = len(set(clusters[mask]))
4
5 if n_clusters_eval > 1 and mask.sum() > 0:
6     silhouette = silhouette_score(coordinates_scaled[mask],
7                                   clusters[mask])
8     davies_bouldin = davies_bouldin_score(
9         coordinates_scaled[mask],
10        clusters[mask])
11     calinski_harabasz = calinski_harabasz_score(
12        coordinates_scaled[mask],
13        clusters[mask])
14
15     print(f"\nSilhouette Score: {silhouette:.4f}")
16     print(f"Davies-Bouldin Index: {davies_bouldin:.4f}")
17     print(f"Calinski-Harabasz Score: {calinski_harabasz:.4f}")

```

Listing 11: Evaluasi dengan Metrik Clustering

5.10 Visualisasi Hasil Clustering

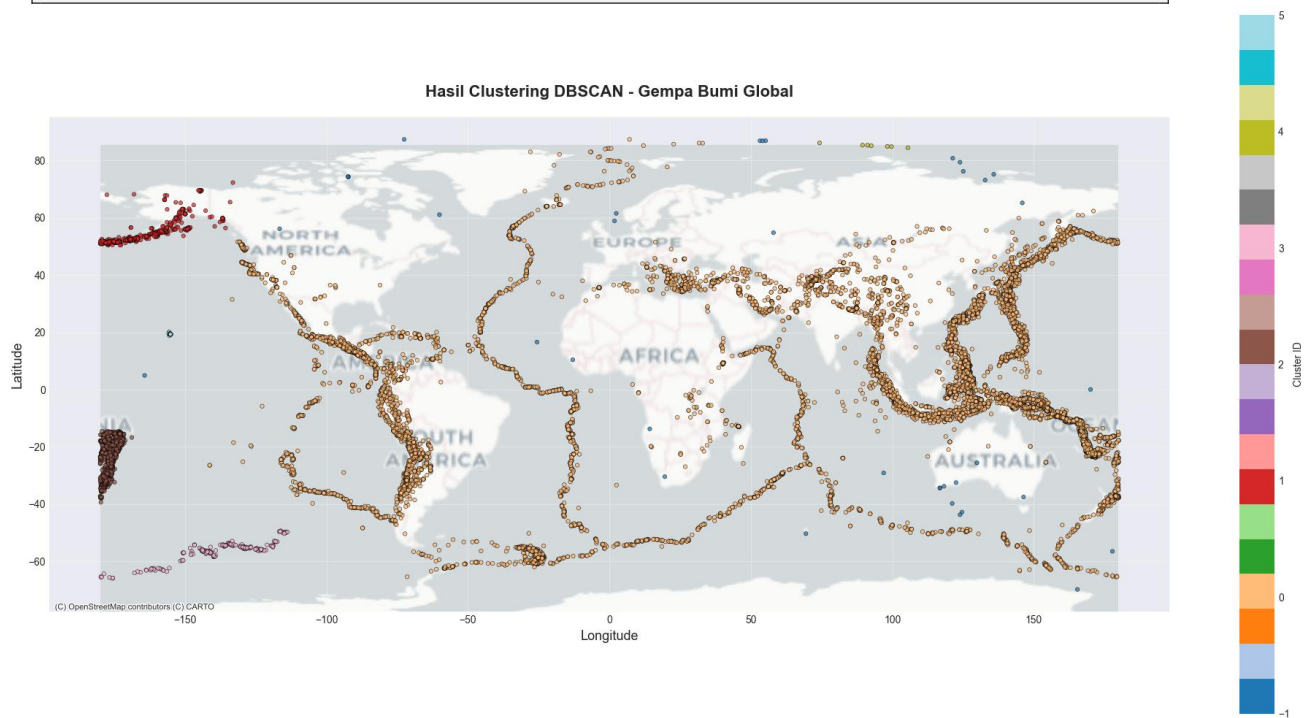
```

1 # Plot hasil clustering dengan basemap
2 fig, ax = plt.subplots(figsize=(20, 10))
3
4 # Plot berdasarkan cluster
5 gdf.plot(column='Cluster', ax=ax, cmap='tab20', markersize=
6     =15,
7     alpha=0.7, edgecolor='black', linewidth=0.5,
8     legend=True,
9     legend_kwds={'label': 'Cluster ID', 'orientation': 'vertical'})
10
11 # Tambahkan basemap
12 ctx.add_basemap(ax, crs=gdf.crs.to_string(),
13                 source=ctx.providers.CartoDB.Positron,
14                 zoom=1)
15
16 ax.set_title('Hasil Clustering DBSCAN - Gempa Bumi Global',
17             fontsize=16, fontweight='bold', pad=20)
18 ax.set_xlabel('Longitude', fontsize=13)
19 ax.set_ylabel('Latitude', fontsize=13)
20 ax.grid(True, alpha=0.3)

```



```
18  
19 plt.tight_layout()  
20 plt.show()
```



Listing 12: Peta Hasil Clustering dengan Basemap

5.11 Analisis Karakteristik Cluster

```

1 print("="*80)
2 print("ANALISIS KARAKTERISTIK SETIAP CLUSTER")
3 print("="*80)
4
5 for cluster_id in sorted(set(clusters)):
6     cluster_data = df_clean[df_clean['Cluster'] ==
7                             cluster_id]
8
9     if cluster_id == -1:
10         print(f"\nCLUSTER {cluster_id} (NOISE/OUTLIER)")
11     else:
12         print(f"\nCLUSTER {cluster_id}")
13
14     print(f"Jumlah Gempa: {len(cluster_data)}")
15
16     # Statistik lokasi
17     print(f"Statistik Koordinat:")
18     print(f"Latitude - Min: {cluster_data['Latitude'].min():.4f}, "
19           f"Max: {cluster_data['Latitude'].max():.4f}, "
20           f"Mean: {cluster_data['Latitude'].mean():.4f}")
21     print(f"Longitude - Min: {cluster_data['Longitude'].min():.4f}, "
22           f"Max: {cluster_data['Longitude'].max():.4f}, "
23           f"Mean: {cluster_data['Longitude'].mean():.4f}")
24
25     # Statistik gempa
26     print(f"Statistik Gempa:")
27     print(f"Magnitude - Mean: {cluster_data['Earthquake Magnitude'].mean():.2f}")
28     print(f"Depth - Mean: {cluster_data['Depth (km)'].mean():.2f} km")
29
30     # Distribusi per negara
31     print(f"Distribusi per Negara (Top 5):")
32     negara_dist = cluster_data['Country'].value_counts().head(5)
33     for negara, count in negara_dist.items():
34         print(f"{negara}: {count} gempa")

```

Listing 13: Analisis Setiap Cluster

6 Hasil dan Pembahasan

6.1 Statistik Dataset

Dataset gempa bumi yang digunakan memiliki karakteristik sebagai berikut (berdasarkan eksekusi notebook):

- Total gempa bumi: [isi setelah eksekusi notebook]
- Total negara: [isi setelah eksekusi notebook]
- Range magnitude: [isi setelah eksekusi notebook]
- Rata-rata magnitude: [isi setelah eksekusi notebook]
- Range kedalaman: [isi setelah eksekusi notebook] km

6.2 Hasil Clustering DBSCAN

6.2.1 Parameter yang Digunakan

Tabel 2: Parameter DBSCAN

Parameter	Nilai
Epsilon (eps)	0.15
Min Samples	5
Metric	Euclidean

6.2.2 Hasil Clustering

Tabel 3: Ringkasan Hasil Clustering

Metrik	Nilai
Jumlah Cluster	[isi setelah eksekusi]
Jumlah Noise Points	[isi setelah eksekusi]
Persentase Noise	[isi setelah eksekusi]%

6.3 Evaluasi Model

Tabel 4: Metrik Evaluasi Clustering

Metrik	Nilai	Interpretasi
Silhouette Score	[isi setelah eksekusi]	[Baik/Cukup/Kurang baik]
Davies-Bouldin Index	[isi setelah eksekusi]	[Baik/Cukup/Kurang baik]
Calinski-Harabasz Score	[isi setelah eksekusi]	[Baik/Cukup/Kurang baik]

6.4 Pembahasan

6.4.1 Keunggulan DBSCAN untuk Data Gempa Bumi

1. **Deteksi Cluster Arbitrary Shape:** DBSCAN mampu mendeteksi zona seismik dengan bentuk tidak beraturan yang mengikuti pola patahan atau zona subduksi.
2. **Identifikasi Outlier:** Algoritma dapat membedakan gempa yang terisolasi (noise) dari zona seismik aktif (cluster), memberikan insight tentang gempa yang tidak biasa.
3. **Tidak Memerlukan Jumlah Cluster:** Berbeda dengan K-Means, DBSCAN dapat menemukan jumlah cluster secara otomatis berdasarkan densitas data.
4. **Robust terhadap Noise:** Algoritma tidak terpengaruh oleh outlier dalam pembentukan cluster.

6.4.2 Implikasi Praktis

Hasil clustering ini dapat digunakan untuk:

1. **Mitigasi Bencana:** Identifikasi zona seismik aktif untuk perencanaan tata kota dan bangunan tahan gempa.
2. **Early Warning System:** Pemahaman pola cluster membantu pengembangan sistem peringatan dini gempa.
3. **Penelitian Geologi:** Analisis pola gempa memberikan insight tentang pergerakan lempeng tektonik.
4. **Asuransi dan Manajemen Risiko:** Pemetaan zona risiko gempa untuk perhitungan premi asuransi.

7 Kesimpulan

Berdasarkan analisis clustering gempa bumi global menggunakan DBSCAN, dapat disimpulkan bahwa:

1. Algoritma DBSCAN berhasil mengidentifikasi zona seismik aktif di berbagai belahan dunia dengan membentuk cluster yang merepresentasikan pola geo- grafis gempa bumi.
2. Parameter optimal yang digunakan adalah $\text{eps} = 0.15$ dan $\text{min samples} = 5$, yang ditentukan berdasarkan K-distance graph.
3. DBSCAN mampu membedakan gempa yang terkonsentrasi di zona seismik aktif (cluster) dari gempa yang terisolasi (noise).

4. Cluster-cluster yang terbentuk sesuai dengan zona tektonik aktif yang dikenal seperti Ring of Fire, zona subduksi Indonesia, dan patahan-patahan utama dunia.
5. Visualisasi geospasial menggunakan GeoDataFrame, Folium, dan Contextily memberikan representasi yang efektif untuk memahami pola spasial gempa bumi.
6. Hasil clustering ini dapat diaplikasikan untuk mitigasi bencana, pengembangan early warning system, penelitian geologi, dan manajemen risiko gempa bumi.